

HyDE: Hypothetical Document Embeddings

A Detailed Guide to Zero-Shot Dense Retrieval Without Relevance Labels

Prepared for Sourav Bhattacharya

Created: July 10, 2026, 20:25 UTC

Contents

1. What Is HyDE?	2
1.1 The Problem HyDE Solves	2
2. Background: Standard Dense Retrieval	3
3. How HyDE Works	4
4. Why It Works: The Vector-Space Intuition	5
5. A Worked Example	6
6. Implementing HyDE in Python	7
6.1 Minimal Implementation from Scratch	7
6.2 Using LangChain's Built-In HyDE Chain	8
6.3 Full Retrieval Pipeline with a Vector Database	9
7. Benchmark Results	10
8. Strengths and Limitations	11
8.1 Strengths	11
8.2 Limitations	11
9. When to Use HyDE	12
10. Related Techniques	13
11. Summary	14
References	15

1. What Is HyDE?

HyDE (Hypothetical Document Embeddings) is a retrieval technique that improves search by having a large language model write a **fake, “hypothetical” answer document** for a query first, and then searching a corpus using the embedding of that fake document instead of the embedding of the original query. It was introduced in the 2022 paper *“Precise Zero-Shot Dense Retrieval without Relevance Labels”* by Gao, Ma, Lin, and Callan, and later published at ACL 2023.

The core idea sounds almost backwards at first: instead of matching a question against documents, HyDE asks an LLM to *imagine* what a good answer document would look like, and matches documents against that imagined answer instead. The generated document may contain factual errors or outright hallucinations — that is expected and, perhaps surprisingly, does not matter much. What matters is that the hypothetical document captures the *style, vocabulary, and structure* of a real answer, which turns out to be a much better search query than the original question text.

1.1 The Problem HyDE Solves

Dense retrieval systems typically encode a query and a set of candidate documents into vectors using the same (or a paired) encoder, then rank documents by vector similarity (usually cosine similarity). This works well when the system has been fine-tuned on labeled query-document pairs for the target domain. Two problems show up in practice:

1. **Labeled data is expensive and often unavailable.** Building a good dense retriever traditionally requires large sets of (query, relevant document) pairs, which are costly to collect and rarely cover every domain or language a system might need.
2. **Queries and documents don’t “look” alike.** A query is often a short question (“What is Milvus?”), while a relevant document is a longer, differently structured passage (“Milvus is an open-source vector database...”). Even a well-trained encoder can struggle to place these two very different text styles close together in embedding space — an issue often called the **query-document asymmetry gap**.

HyDE sidesteps both problems: it needs no labeled query-document pairs (hence “zero-shot”), and it closes the style gap by first turning the query into something that already looks like a document.

2. Background: Standard Dense Retrieval

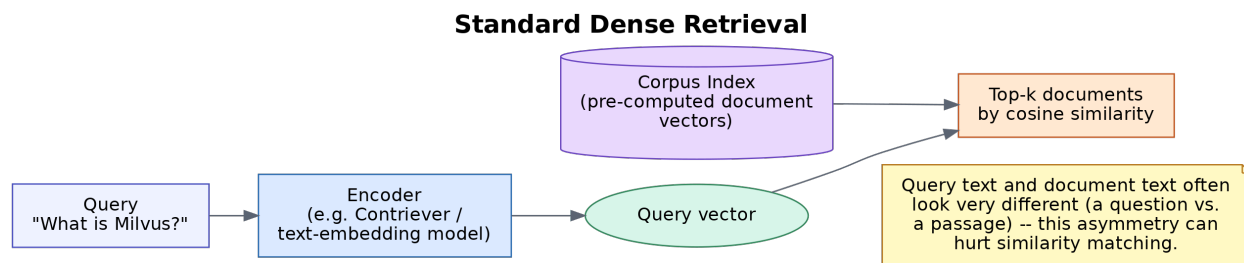


Figure 1: In standard dense retrieval, the query is embedded directly and compared against pre-computed document vectors.

In the conventional setup, a query is passed through an encoder (for example, Contriever, or a commercial embedding model) to produce a single query vector. This vector is compared against a pre-computed index of document vectors using approximate nearest neighbor (ANN) search, and the closest documents are returned.

This approach is fast and simple, and works reasonably well when the encoder has seen similar queries and documents during training. But encoders trained mostly on document-document or passage-passage similarity can struggle when asked to match a short question against a long passage — the two text types occupy noticeably different regions of the embedding space even when they are topically related.

3. How HyDE Works

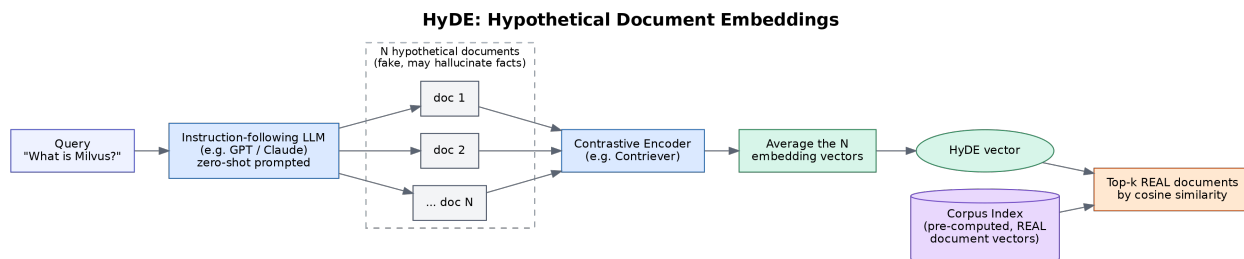


Figure 2: HyDE generates several hypothetical documents with an LLM, embeds and averages them, and searches the REAL corpus index with that averaged vector.

HyDE replaces the single “encode the query” step with a short generate-then-embed pipeline:

1. **Generate.** An instruction-following LLM (the original paper used InstructGPT; today this could just as well be GPT-4-class or Claude models) is zero-shot prompted with something like: “*Write a passage that answers the question: {query}.*” This is repeated N times (the paper and most implementations use $N = 5$), each generation sampled with some temperature so the documents vary slightly.
2. **Embed.** Each of the N hypothetical documents is encoded into a vector using an unsupervised contrastively-trained encoder (the paper used Contriever; production systems often substitute any general-purpose embedding model).
3. **Average.** The N embedding vectors are averaged into a single vector. Averaging smooths out idiosyncratic hallucinations in any one generation while reinforcing the parts of the answer that are consistently relevant across samples.
4. **Search.** This averaged “HyDE vector” is used exactly like a normal query vector: it is compared against the **real**, pre-computed document embeddings in the corpus index, and the top- k most similar real documents are returned.

Crucially, the LLM’s hallucinations never enter the final results — the generated documents are only ever used to produce an embedding. The actual documents returned to the user are always real, verified corpus entries.

4. Why It Works: The Vector-Space Intuition

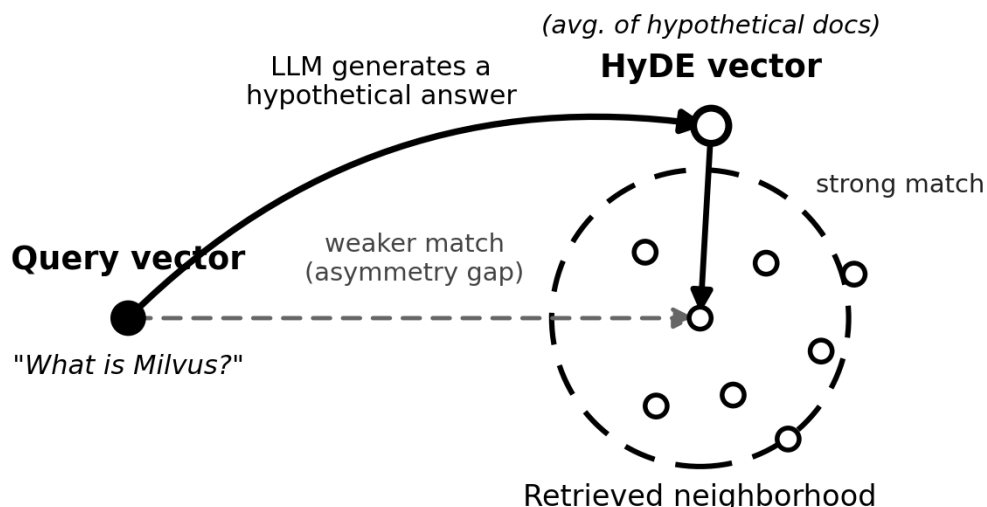


Figure 3: The raw query vector sits further from the real document cluster than the HyDE vector does, because the HyDE vector already looks document-shaped.

The intuition behind HyDE is a geometric one. A short question and a long passage that answers it are semantically related but stylistically different, so their embeddings may sit further apart in vector space than two passages that both “look like” documents. By asking an LLM to draft an answer first, HyDE produces a document-shaped piece of text whose embedding is much more likely to land inside the neighborhood of genuinely relevant real documents — even though the drafted text itself is never shown to the user and may contain factual mistakes.

This also explains why HyDE tends to help most on **out-of-domain or low-resource retrieval tasks**: the LLM’s broad world knowledge fills in a plausible-sounding, document-shaped answer even for topics the retrieval encoder itself was never fine-tuned on.

5. A Worked Example

Consider a small corpus of four unrelated facts, and the query “What is Milvus?”:

Corpus:

1. It usually takes between 30 minutes and two hours to remove a wisdom tooth.
2. The COVID-19 pandemic significantly impacted mental health, increasing depression and anxiety.
3. Humans have used fire for approximately 800,000 years.
4. Milvus is a cloud-based database for vector storage.

A direct embedding of the four-word query “What is Milvus?” must compete against four full sentences using only that short phrase. HyDE instead prompts an LLM: *“Write a passage that answers the question: What is Milvus?”*, which might produce something like:

Hypothetical document:

"Milvus is an open-source vector database designed for similarity search on massive-scale embedding data. It is commonly used to power retrieval-augmented generation and recommendation systems..."

This generated passage is now stylistically similar to corpus document 4 — both are declarative sentences describing what Milvus is — so their embeddings land close together, and document 4 is correctly retrieved even though the LLM’s specific phrasing (“open-source”, “massive-scale”) does not literally appear in the real corpus entry.

6. Implementing HyDE in Python

6.1 Minimal Implementation from Scratch

The following mirrors the pipeline in Section 3 using the OpenAI API for both generation and embeddings, and a plain in-memory corpus for clarity:

```
import numpy as np
from openai import OpenAI

client = OpenAI()

corpus = [
    "It usually takes between 30 minutes and two hours to "
    "remove a wisdom tooth.",
    "The COVID-19 pandemic significantly impacted mental "
    "health, increasing depression and anxiety.",
    "Humans have used fire for approximately 800,000 years.",
    "Milvus is a cloud-based database for vector storage.",
]

def embed(texts, model="text-embedding-3-small"):
    resp = client.embeddings.create(input=texts, model=model)
    return np.array([d.embedding for d in resp.data])

def generate_hypothetical_docs(query, n=5):
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system",
             "content": "Write a short passage that answers "
                        "the question."},
            {"role": "user", "content": query},
        ],
        n=n,
        temperature=0.7,
        max_tokens=150,
    )
    return [choice.message.content for choice in resp.choices]

def hyde_vector(query, n=5):
    docs = generate_hypothetical_docs(query, n=n)
    doc_vecs = embed(docs)
    return doc_vecs.mean(axis=0)
```

```
def hyde_search(query, corpus_vecs, top_k=3):
    q_vec = hyde_vector(query)
    sims = corpus_vecs @ q_vec / (
        np.linalg.norm(corpus_vecs, axis=1) * np.linalg.norm(q_vec)
    )
    ranked = np.argsort(-sims)[:top_k]
    return [(corpus[i], float(sims[i])) for i in ranked]

corpus_vecs = embed(corpus)
results = hyde_search("What is Milvus?", corpus_vecs)
for text, score in results:
    print(f"{score:.3f} {text}")
```

6.2 Using LangChain's Built-In HyDE Chain

LangChain ships a ready-made HypotheticalDocumentEmbedder that wraps the same generate-embed-average pipeline (it currently lives under the legacy langchain_classic chains module in newer LangChain releases, alongside a manual LCEL-style implementation being the more commonly recommended approach today):

```
from langchain.chains import HypotheticalDocumentEmbedder
from langchain.chains.llm import LLMChain
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain.prompts import PromptTemplate

base_embeddings = OpenAIEmbeddings()
llm = ChatOpenAI(temperature=0.7)

prompt = PromptTemplate(
    input_variables=["question"],
    template="Write a short passage that answers the "
            "question.\nQuestion: {question}\nPassage:",
)
llm_chain = LLMChain(llm=llm, prompt=prompt)

hyde_embeddings = HypotheticalDocumentEmbedder(
    llm_chain=llm_chain,
    base_embeddings=base_embeddings,
)

# Produces one averaged embedding, ready to hand to any
# vector store's similarity_search_by_vector() method.
hyde_vec = hyde_embeddings.embed_query("What is Milvus?")
```


6.3 Full Retrieval Pipeline with a Vector Database

In production, the pre-computed corpus embeddings live in a vector database (Milvus, Qdrant, pgvector, and similar) rather than a Python list. Only the query side changes from a standard pipeline:

```
from pymilvus import MilvusClient

client_db = MilvusClient("hyde_demo.db")
client_db.create_collection(collection_name="docs", dimension=1536)

# One-time corpus indexing (standard embeddings, no HyDE needed)
vectors = embed(corpus)
client_db.insert(collection_name="docs", data=[
    {"id": i, "vector": vectors[i].tolist(), "text": corpus[i]}
    for i in range(len(corpus))
])

# At query time: build the HyDE vector, then search normally
query_vec = hyde_vector("What is Milvus?")
hits = client_db.search(
    collection_name="docs",
    data=[query_vec.tolist()],
    limit=3,
    output_fields=["text"],
)
for hit in hits[0]:
    print(hit["distance"], hit["entity"]["text"])
```

Note that the corpus side of the pipeline is completely unchanged from standard dense retrieval — HyDE only changes how the *query* is turned into a vector.

7. Benchmark Results

The original paper evaluated HyDE across web search, question answering, and fact verification benchmarks (using BEIR-style datasets) with these broad findings:

- **HyDE consistently outperforms BM25 and unsupervised Contriever** across most datasets and metrics, including nDCG@10 and recall.
- **HyDE remains competitive with fine-tuned dense retrievers** on richly supervised benchmarks such as TREC DL19/DL20, despite using no labeled training data at all.
- **Gains extend to multiple languages**, with reported improvements over multilingual Contriever (mContriever) baselines in languages including Korean and Japanese.
- **Gains are largest in low-resource and out-of-domain settings** — exactly where traditional dense retrievers, which rely on in-domain fine-tuning, tend to struggle most.

8. Strengths and Limitations

8.1 Strengths

- **Zero-shot:** no relevance-labeled query-document pairs are required to build or adapt the retriever.
- **Domain and language agnostic:** works across web search, biomedical QA, fact verification, and multiple languages without retraining the encoder.
- **Bridges the asymmetry gap:** turns a short, question-shaped query into a document-shaped piece of text before embedding, which tends to match the corpus better (Section 4).
- **Composable:** HyDE only changes how the query is embedded; it can be dropped into any existing dense retrieval or RAG pipeline without touching the corpus index.

8.2 Limitations

- **Knowledge bottleneck / hallucination risk:** if the LLM knows nothing about a topic, it may generate a confidently wrong hypothetical document, which can point the search in the wrong direction. HyDE inherits the blind spots of whichever LLM generates the hypothetical documents.
- **Extra latency and cost:** each query now requires N additional LLM generations (commonly $N = 5$) before the embedding and search steps even begin, which adds noticeable latency and API cost compared to a single embedding call.
- **Multilingual saturation:** the paper notes that small contrastive encoders can saturate as the number of languages scales, while the generative LLM side can be under-trained for lower-resource languages — the two components’ language coverage does not always line up.
- **No free lunch on well-resourced, in-domain tasks:** when high-quality labeled data already exists for the exact domain, a properly fine-tuned dense retriever can still outperform HyDE; the technique’s main advantage is in zero-shot and out-of-domain conditions.

9. When to Use HyDE

HyDE is a strong candidate when:

- No relevance-labeled data exists for the target domain, and fine-tuning a retriever is not practical.
- The corpus is out-of-domain relative to what the embedding model was originally trained on.
- Queries are short, vague, or phrased very differently from how answers are written (classic question-vs-passage asymmetry).
- The application already has LLM calls in its budget (for example, a RAG system that will call an LLM anyway for the final answer), so one extra generation step is a small relative cost.

Standard dense retrieval (or a fine-tuned retriever) is usually the better choice when:

- Strict low-latency requirements make an extra LLM generation step per query unacceptable.
- Ample labeled in-domain data already exists to fine-tune a retriever directly.
- Query volume is high enough that the added LLM cost of generating hypothetical documents becomes a significant expense.

10. Related Techniques

HyDE sits within a broader family of query-transformation methods for retrieval:

- **Classical pseudo-relevance feedback (PRF)**, such as the Rocchio algorithm, expands a query using terms from the top initially-retrieved documents — HyDE can be seen as an LLM-powered, generative descendant of this decades-old idea.
- **Query expansion / rewriting** uses an LLM to reformulate or add detail to a query without generating a full hypothetical answer document.
- **Step-back prompting** asks an LLM to generate a more general, abstracted version of a question before retrieval, which can be combined with HyDE-style generation.
- **RAG-Fusion** generates multiple *reformulated queries* (rather than hypothetical answers) and merges the ranked results from each, addressing a related but distinct problem.

11. Summary

HyDE reframes retrieval as generate-then-embed rather than embed-then-search: an LLM first drafts one or more hypothetical answer documents for a query, those drafts are embedded and averaged, and the resulting vector is used to search a corpus of real documents exactly as a normal query embedding would be. Because the hallucinated content in the draft never reaches the user — only its embedding does — HyDE gets the benefit of the LLM’s broad knowledge and document-shaped phrasing without inheriting its factual errors in the final results. This makes it a particularly effective, genuinely zero-shot technique for out-of-domain, low-resource, or otherwise hard-to-label retrieval settings, at the cost of extra per-query LLM calls.

References

1. Gao, L., Ma, X., Lin, J., and Callan, J. *Precise Zero-Shot Dense Retrieval without Relevance Labels*. arXiv:2212.10496. <https://arxiv.org/abs/2212.10496>
2. ACL Anthology. *Precise Zero-Shot Dense Retrieval without Relevance Labels* (ACL 2023). <https://aclanthology.org/2023.acl-long.99/>
3. Hugging Face Papers. *Precise Zero-Shot Dense Retrieval without Relevance Labels*. <https://huggingface.co/papers/2212.10496>
4. Zilliz Learn. *Improving Information Retrieval and RAG with Hypothetical Document Embeddings (HyDE)*. <https://zilliz.com/learn/improve-rag-and-information-retrieval-with-hyde-hypothetical-document-embeddings>
5. Haystack Documentation. *Hypothetical Document Embeddings (HyDE)*. <https://docs.haystack.deepset.ai/docs/hypothetical-document-embeddings-hyde>
6. LangChain API Reference. *HypotheticalDocumentEmbedder*. https://python.langchain.com/api_reference/langchain/chains/langchain.chains.hyde.base.HypotheticalDocumentEmbedder.html
7. Thakur, N. et al. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. <https://arxiv.org/abs/2104.08663>